PUC Minas

INGESTÃO E INTEGRAÇÃO DE DADOS

Prof. Victor Sales Silva
victorssbh@hotmail.com

Na aula de hoje...

- Estratégias de ingestão de dados
- Integração de APIs, bancos relacionais, NoSQL e arquivos
- Processamento de dados streaming e CDC
- Carga completa e incremental
- Padronização e integração de múltiplas fontes
- Prática

VAMOS COMEÇAR?

RESUMO

- Objetivos da aula
  - Estratégias de ingestão de dados
  - Integração de APIs, bancos relacionais, NoSQL e arquivos
  - Processamento de dados Streaming e CDC
  - Carga completa e incremental
  - Padronização e integração de múltiplas fontes
  - Construção de um processo de ingestão na prática

Estratégias de ingestão de dados

- A ingestão é o processo responsável por obter dados de uma fonte e disponibilizá-los para o pipeline.



- As fontes podem possuir características muito diferentes.

Estratégias de ingestão de dados

FONTE	EXEMPLO	DESAFIOS E CARACTERÍSTICAS GERAIS
Banco relacional	PostgreSQL, SQL Server	Transações ACID, schemas rígidos, tabelas relacionais
NoSQL	MongoDB, Cassandra	Estrutura documental flexível, schemas dinâmicos
API	REST, GraphQL	Sujeito a paginação, limites de taxa (rate limit) e timeouts
Arquivos	CSV, JSON, XML, Parquet	Diferentes encodings, delimitadores, chegada assíncrona
Eventos	Cliques, vendas, sensores	Fluxos contínuos de altíssima frequência e baixa latência
Sistemas externos	Parceiros, fornecedores	Falta de controle direto sobre a infraestrutura da fonte
Aplicações	Sistemas transacionais	Necessidade de não causar impacto no desempenho (lock/overhead)
CDC	Bancos de dados	Alterações realizadas em bancos de dados.

Estratégias de ingestão de dados

Ingestão x Transformação

INGESTÃO (Porta de Entrada)

- Acessar a fonte de dados com resiliência
- Extrair os dados sem alterar seu conteúdo original
- Controlar e monitorar a frequência da coleta
- Registrar metadados técnicos de execução
- Preservar a fidelidade dos dados recebidos
- Disponibilizar os dados brutos no Data Lake

TRANSFORMAÇÃO (Processamento)

- Limpar e tratar inconsistências/valores nulos
- Padronizar tipos de dados, nomes e formatos
- Integrar e cruzar informações de múltiplas fontes
- Agregar dados em diferentes granularidades
- Enriquecer o dataset com dados de contexto
- Aplicar regras de negócio específicas para consumo

Estratégias de ingestão de dados

- A ingestão precisa considerar:



💡 **Diretriz da Arquitetura de Ingestão**

A análise sequencial dessas 6 dimensões determina a escolha da ferramenta, do padrão de arquitetura (Batch, CDC ou Streaming) e da infraestrutura necessária para o pipeline de dados.

Estratégias de ingestão de dados

Fatores Quantitativos: Volume e Frequência

📦 VOLUME

Quanto dado precisa ser transportado?

- **10 MB por dia:** Uma estratégia simples pode ser suficiente.
- **500 GB por dia:** Exige maior eficiência e controle.
- **Terabytes:** Processamento distribuído e ingestão incremental tornam-se importantes.

❓ **Pergunta-chave:**
Quanto dado será movimentado a cada execução?

🔄 FREQUÊNCIA

Com que frequência novos dados são produzidos?

- ➔ **Uma vez por mês**
- ➔ **Diariamente**
- ➔ **A cada hora**
- ➔ **A cada minuto**
- ➔ **Continuamente**

💡 **Exemplo de Negócio:**
Um relatório financeiro pode ser atualizado diariamente, enquanto eventos de navegação podem chegar continuamente.

Estratégias de ingestão de dados

Fatores Quantitativos: Volume

Escalas de Volume Comuns

- • 10 MB por dia (Carga leve)
- • 2 GB por hora (Carga média)
- • 500 GB por dia (Grande volume)
- • 5 TB por semana (Escala massiva)

Impactos Diretos no Pipeline

- • Tempo total de processamento e extração
- • Dimensionamento do armazenamento de destino
- • Consumo de largura de banda de rede
- • Custos de computação e infraestrutura
- • Seleção da estratégia de carga (Full vs Incremental)
- • Necessidade de particionamento de dados

Exemplo Prático de Decisão

Uma tabela transacional com 500 milhões de registros dificilmente deveria ser extraída integralmente todos os dias sem uma justificativa técnica ou de negócio muito clara.

Estratégias de ingestão de dados

Fatores Quantitativos: Frequência

Eventual

Uma única carga ou migração pontual de dados.

Diária

Fechamento financeiro diário ou consolidação de campanhas.

Horária

Acompanhamento de indicadores operacionais intraday.

Minutos

Monitoramento de métricas quase em tempo real.

Contínua

Processamento ininterrupto de eventos, sensores e transações.

Estratégias de ingestão de dados

Estrutura e Acesso: Formato e Disponibilidade

📄 FORMATO

Como os dados são disponibilizados?

- CSV / Excel
- JSON / XML
- Parquet
- Tabelas relacionais
- Documentos NoSQL
- Eventos

📌 **Impacto:**
O formato influencia diretamente a forma de leitura, transformação e armazenamento do pipeline.

🌐 DISPONIBILIDADE

A fonte está disponível quando o pipeline precisa acessá-la?

Cenários comuns em APIs e Fontes Externas:

⚠️ Ficar indisponível repentinamente

⚠️ Limitar requisições (Rate Limiting)

⚠️ Exigir autenticação complexa / rotação de chaves

⚠️ Apresentar janelas periódicas de manutenção

Uma estratégia de ingestão precisa antecipar e mitigar esses cenários.

Estratégias de ingestão de dados

Estrutura e Acesso: Formato

Estruturados

Tabelas relacionais (SQL), arquivos CSV, Parquet. Possuem esquemas bem definidos e tipos rigorosos.

Semiestruturados

Arquivos JSON, XML, documentos NoSQL. Possuem estrutura flexível, campos opcionais e aninhados.

Eventos

Mensagens de streaming, eventos em JSON, logs de aplicações. Chegada contínua e assíncrona.

Estratégias de ingestão de dados

Estrutura e Acesso: Disponibilidade

Possíveis Problemas na Origem

- Sistema de origem fora do ar ou em manutenção
- Janelas de manutenção programadas
- Timeouts de conexão e limite de requisições
- Autenticação e chaves de acesso expiradas
- Indisponibilidade momentânea de rede
- Arquivo diário que ainda não foi disponibilizado

Requisitos do Pipeline de Ingestão

- Retry (tentativas automáticas de reexecução)
- Timeout (tempo máximo de espera configurado)
- Logs detalhados de falhas e auditoria
- Controle rígido do estado de execução
- Suporte a reprocessamento idempotente

Estratégias de ingestão de dados

Requisitos Operacionais: Latência e Confiabilidade

⚡ LATÊNCIA

Quanto tempo pode passar entre a geração e o processamento do dado?

- **Baixa latência:** segundos ou poucos minutos.
- **Alta latência aceitável:** horas ou dias.

💡 **Exemplo Comparativo:**

🚨 *Fraude: segundos podem ser críticos para bloquear uma transação.*
📊 *Relatório mensal: algumas horas de espera são perfeitamente aceitáveis.*

🛡️ CONFIABILIDADE

Podemos confiar que os dados serão entregues corretamente?

Pontos de atenção e falhas prováveis:

- Falhas de conexão durante a transferência
- Dados duplicados / Re-tentativas
- Registros perdidos ou corrompidos
- Arquivos incompletos
- Alterações inesperadas na estrutura (Schema Drift)
- Necessidade de reprocessamento (Repipelining)

🔑 **Princípio Fundamental:**
Uma ingestão confiável deve ser projetada para detectar, registrar e tratar falhas automaticamente.

Estratégias de ingestão de dados

Requisitos Operacionais: Latência

Alta Latência Aceitável (Batch tradicional)

• • Exemplo: Relatório financeiro consolidado diário.

• • O processamento pode ocorrer de madrugada (ex: 02:00) sem prejuízo ao negócio.

• • Foco em eficiência de lote e consumo otimizado de recursos.

Baixa Latência Necessária (Quase Tempo Real / Streaming)

• • Exemplo: Detecção de fraude em cartão ou recomendação instantânea.

• • O dado precisa estar disponível em segundos ou poucos minutos para ação imediata.

• • Streaming significa dados tratados de forma contínua ou em micro-lotes.

Estratégias de ingestão de dados

Requisitos Operacionais: Confiabilidade

Riscos e Erros Comuns na Ingestão

- Registros duplicados ou perdidos no caminho
- Recebimento de arquivos incompletos ou corrompidos
- Conexão interrompida no meio da transferência
- API retornando dados parciais ou despaginados
- Registros chegando fora da ordem cronológica
- Alteração não comunicada de schema na origem

Mecanismos de Controle Essenciais

- Nome da fonte / sistema de origem
- Data, hora e timestamp da coleta
- Quantidade total de registros extraídos
- Período exato dos dados coletados
- Status de sucesso ou falha da execução
- Identificador único da execução (batch_id)
- Detalhamento de erros ocorridos

Estratégias de ingestão de dados

Cenários de ingestão de dados

- Não existe uma única forma de ingerir dados.

CARGA COMPLETA
Copia todo o conjunto de dados.
Exemplo: carregar novamente uma tabela com 10 milhões de registros.
Quando usar: pequenos volumes, carga inicial ou reconstrução do destino.

CARGA INCREMENTAL
Copia somente os dados novos ou modificados.
Exemplo: buscar apenas pedidos alterados desde a última execução.
Quando usar: grandes volumes ou processos executados frequentemente.

STREAMING
Processa dados conforme novos eventos são produzidos.
Exemplo: cada venda gera um evento que pode ser processado imediatamente.
Quando usar: baixa latência e acompanhamento contínuo.

CDC
Captura as alterações realizadas na origem.
Exemplo: identificar INSERT, UPDATE e DELETE realizados em uma tabela.
Quando usar: replicação, integração contínua e sincronização de dados.

Estratégias de ingestão de dados

Cenários de ingestão de dados: Carga completa

Fonte de Dados

Exemplo: Tabela 'clientes' com 100.000 registros na origem.

Extração Total

Lê e transporta todos os 100.000 registros a cada execução agendada.

Destino / Data Lake

Sobreve/substitui o destino ou grava um novo snapshot completo do conjunto.

Cenários Adequados

Primeira carga, tabelas pequenas, dados sem controle de data/alteração, reprocessamentos.

CENÁRIO CRÍTICO

Executar 'SELECT * FROM pedidos' todos os dias em uma tabela com 500 milhões de registros.

PRINCIPAIS PROBLEMAS IDENTIFICADOS

- Alto volume desnecessário de dados transferidos na rede
- Maior tempo total de execução da janela de carga
- Consumo excessivo de memória e processamento
- Alto impacto e lentidão no banco de dados de origem (locks)
- Aumento significativo de custos de infraestrutura e nuvem
- Ineficiência ao reprocessar dados que não sofreram modificação

REFLEXÃO CENTRAL

Se apenas 20 mil registros mudaram no dia, por que transportar 500 milhões? Essa é a motivação para a Carga Incremental.

Estratégias de ingestão de dados

Cenários de ingestão de dados: Carga completa

Cenário Crítico

Executar 'SELECT * FROM pedidos' todos os dias em uma tabela com 500 milhões de registros.

Principais Problemas Identificados

- Alto volume desnecessário de dados transferidos na rede
- Maior tempo total de execução da janela de carga
- Consumo excessivo de memória e processamento
- Alto impacto e lentidão no banco de dados de origem (locks)
- Aumento significativo de custos de infraestrutura e nuvem
- Ineficiência ao reprocessar dados que não sofreram modificação

Reflexão Central

Se apenas 20 mil registros mudaram no dia, por que transportar 500 milhões? Essa é a motivação para a Carga Incremental.

Estratégias de ingestão de dados

Escolha da estratégia

CENÁRIO	ESTRATÉGIA COMUM
Carga inicial de uma tabela	Completa
Tabela grande atualizada diariamente	Incremental
Arquivos gerados diariamente	Incremental
Eventos de vendas	Streaming
Alterações em banco transacional	CDC
Reconstrução de um Data Lake	Completa

EXEMPLO

Uma tabela de 100 milhões de pedidos não precisa ser copiada inteira todos os dias.

Se apenas 50 mil pedidos foram alterados:

- **Carga completa → 100 milhões de registros**
- **Carga incremental → 50 mil registros**

💡 **COMO ESCOLHER?**

A estratégia depende principalmente de como os dados são produzidos e de quanto tempo podemos esperar por eles.

Estratégias de ingestão de dados

Carga incremental

1

Identificação do Estado

Grava o timestamp/marco da última execução (ex: 10:00:00).

2

Filtro Delta

Filtra e busca na origem somente os dados novos, alterados ou excluídos após 10:00:00.

3

Transferência Reduzida

Transporta um volume significativamente menor de dados pela rede.

4

Vantagens diretas

Menor tempo, menor impacto na origem, menor custo e maior frequência possível.

Estratégias de ingestão de dados

Carga incremental

Controle do marca d'água (Watermark)

1

1. Leitura da Marca Anterior

Consulta da tabela de controle: 'Última execução em 2026-09-21 10:00:00'.

2

2. Execução da Consulta Delta

SELECT * FROM pedidos WHERE data_atualizacao > '2026-09-21 10:00:00';

3

3. Gravação dos Dados Brutos

Persistência do lote incremental coletado na camada Bronze.

4

4. Atualização do Watermark

Novo watermark passa a ser o maior timestamp da coluna 'data_atualizacao' coletada.

Estratégias de ingestão de dados

Carga incremental: INSERT vs UPDATE vs DELETE

Operações Possíveis em uma Tabela

- • INSERT: Novo registro criado no banco
- • UPDATE: Registro existente alterado
- • DELETE: Registro removido fisicamente da origem

Limitação do Controle por ID Sequencial

- • A lógica 'WHERE id > ultimo_id' captura novos INSERTs.
- • Porém, NÃO detecta UPDATEs em registros antigos.
- • Tampouco detecta DELETEs (a chave simplesmente desaparece da origem).
- • Exige o uso de colunas de atualização ou CDC.

Estratégias de ingestão de dados

Streaming

Conceito Fundamental

No processamento tradicional (Batch), os dados são acumulados em lotes para processamento periódico. No Streaming, cada evento é tratado individualmente ou em micro-lotes à medida que é produzido.

Exemplos de Fontes Contínuas

- Vendas e transações de e-commerce em tempo real
- Cliques e navegação de usuários em websites (Clickstream)
- Sensores de IoT, telemetria e geolocalização
- Logs de aplicações e servidores de infraestrutura
- Eventos de redes sociais e notificações

Desafios Técnicos do Streaming

- Tratamento de eventos duplicados por falha de rede
- Gestão de eventos atrasados (Late Data) ou fora de ordem
- Lidar com quedas e reconexões temporárias de infraestrutura
- Filtragem e descarte de mensagens inválidas ou corrompidas
- Evolução indesejada de schema nas mensagens
- Capacidade de reprocessar dados retroativos em caso de erro

Pilares do Pipeline Streaming Robust

- Confiabilidade e garantias de entrega (At-least-once / Exactly-once)
- Observabilidade, métricas e alertas em tempo real
- Gerenciamento de estado (State Management & Checkpointing)
- Estratégias de tratamento de erros e Dead Letter Queues (DLQ)

Estratégias de ingestão de dados

Comparativo: Batch vs Streaming

CARACTERÍSTICA	BATCH (LOTE)	STREAMING (CONTÍNUO)
Natureza dos Dados	Acumulados e delimitados	Contínuos e infinitos
Execução	Periódoca (ex: diária, horária)	Contínua e ininterrupta
Latência	Maior (minutos a horas)	Menor (milissegundos a segundos)
Complexidade	Mais simples de manter	Maior complexidade operacional
Uso Típico	Relatórios e consolidações	Monitoramento ao vivo e alertas

Estratégias de ingestão de dados

CDC (Change Data Capture)

1

1. Operação Transacional

Exemplo: Pedido 1001 sofre um UPDATE (valor altera de R$ 200 para R$ 250).

2

2. Registro no Log da Origem

A alteração é gravada no log de transações do banco (WAL/Binlog).

3

3. Captura do Evento CDC

O mecanismo de CDC lê o log e gera um evento com o estado anterior e novo.

4

4. Sincronização em Tempo Real

O evento é disponibilizado para consumo mantendo o destino perfeitamente espelhado.

Estratégias de ingestão de dados

Comparativo: Carga Incremental vs CDC

CARACTERÍSTICA	CARGA INCREMENTAL	CHANGE DATA CAPTURE (CDC)
Mecanismo	Consulta ativamente a tabela via SQL	Captura passivamente logs/eventos da origem
Dependência	Exige coluna de controle (ex: timestamp)	Independe de colunas na tabela
Detecção de Operações	Captura INSERT e UPDATE (difícil DELETE)	Captura perfeitamente INSERT, UPDATE e DELETE
Complexidade	Mais simples de implementar	Mais complexo (requer infraestrutura dedicada)
Caso de Uso Ideal	Muitos cenários analíticos batch	Sincronização contínua de baixa latência

Integração de APIs, bancos relacionais, NoSQL e arquivos

- Em um ambiente real, os dados necessários para uma análise dificilmente estão em um único lugar.
- A mesma informação pode estar em fontes diferentes.

PostgreSQL
Pedidos e clientes

API
Informações de localização

MongoDB
Interações dos usuários

CSV
Dados de campanhas

Objetivo da Integração

Combinar essas fontes em um conjunto de dados consistente e utilizável.
Integração não significa apenas copiar dados. É necessário entender estruturas, identificadores, tipos e regras de cada fonte.

Integração de APIs, bancos relacionais, NoSQL e arquivos

API

- API são conjunto de regras e padrões que permitem que sistemas troquem dados. Normalmente, através de requisições HTTP/HTTPS.

UMA INGESTÃO DE API PRECISA CONSIDERAR:

- Autenticação e autorização
- Paginação
- Limites de requisições
- Tratamento de erros
- Timeout e novas tentativas
- Controle do período consultado
- Formato da resposta
- Controle de duplicidade

EXEMPLO DE PAGINAÇÃO

Uma API de pedidos disponibiliza:
GET /pedidos?page=1
GET /pedidos?page=2
GET /pedidos?page=3

O processo de ingestão precisa percorrer as páginas até obter todos os registros necessários.

💡 **BOA PRÁTICA**

Registrar informações da execução:

data/hora + origem + quantidade + status + período coletado

Integração de APIs, bancos relacionais, NoSQL e arquivos

Bancos relacionais

- Bancos relacionais organizam os dados principalmente em:
  - Tabelas ➔ Colunas ➔ Registros ➔ Relacionamento

clientes	
id_cliente	nome
cidade	estado

pedidos	
id_pedido	id_cliente
data	valor

NA INGESTÃO, DEVEMOS OBSERVAR:

- Chaves primárias
- Chaves estrangeiras
- Tipos de dados
- Relacionamentos
- Volume das tabelas
- Data de atualização
- Estratégia de carga

EXEMPLO DE FLUXO PARA DATA LAKE

[1] PostgreSQL (Origem)

↓

[2] Ingestão (Processo)

↓

[3] Dados Brutos (Bronze)

↓

[4] Processamento Posterior (Silver/Gold)

Integração de APIs, bancos relacionais, NoSQL e arquivos

NoSQL

- NoSQL não possui uma única estrutura de dados

📄 **Documentos** | 🔑 **Chave-valor** | 🕸️ **Grafos** | 📊 **Colunas**

EXEMPLO — DOCUMENTO JSON

```
{
  "cliente": 125,
  "cidade": "Belo Horizonte",
  "preferencias": {
    "categoria": "eletrônicos"
  }
}
```

⚠️ DESAFIO DE INGESTÃO NOSQL

Diferentemente de uma tabela relacional, os documentos podem apresentar:

- **Campos opcionais**
- **Estruturas aninhadas**
- **Tipos diferentes**
- **Schemas parcialmente variáveis**

🔑 **Diretriz:**
Por isso, a ingestão deve preservar a informação original e permitir tratamento posterior.

Integração de APIs, bancos relacionais, NoSQL e arquivos

Arquivos

- Arquivos continuam sendo uma importante fonte de dados.

FORMATO	USO FREQUENTE
CSV	Integrações simples e exportações
JSON	APIs e dados semiestruturados
Parquet	Analytics e Data Lake
XML	Sistemas legados e integrações

EXEMPLO DE PARTICIONAMENTO / NOMENCLATURA:

- vendas_2026_09_01.csv
- vendas_2026_09_02.csv
- vendas_2026_09_03.csv

A organização dos arquivos também pode fazer parte da estratégia de ingestão.

PROBLEMAS COMUNS EM ARQUIVOS

- ⚠️ Encoding diferente (UTF-8, Latin1, ISO-8859-1)
- ⚠️ Separadores diferentes (vírgula, ponto e vírgula, tab)
- ⚠️ Cabeçalhos inconsistentes
- ⚠️ Colunas ausentes
- ⚠️ Tipos incorretos
- ⚠️ Arquivos duplicados
- ⚠️ Arquivos parcialmente gravados durante a leitura

Processamento de dados streaming e CDC

- Em um processo tradicional, os dados podem ser acumulados e processados posteriormente. No Streaming, os eventos são tratados conforme chegam.

EXEMPLO DE FLUXO EM TEMPO REAL:

Venda realizada

↓

Evento produzido

↓

Processamento imediato

↓

Indicador atualizado

APLICAÇÕES COMUNS:

- Monitoramento
- Fraude
- IoT
- Logs
- Clickstream
- Transações
- Indicadores em tempo real

🧠 **Conceito Importante:**

Streaming não significa necessariamente "instantâneo". Existe uma latência entre a produção, o processamento e o consumo do evento.

Processamento de dados streaming e CDC

CDC – Change Data Capture

- CDC busca identificar alterações ocorridas na origem.

EVENTO	SIGNIFICADO
INSERT	Novo registro
UPDATE	Registro alterado
DELETE	Registro removido

🛠️ **TECNOLOGIAS COMUNS:**

Debezium + Kafka

Uma combinação extremamente comum para capturar alterações de bancos de dados e disponibilizá-las como eventos continuamente.

EXEMPLO PRÁTICO DE CDC

Banco de Dados:
Cliente 101 ➔ Cidade = BH

↓ **UPDATE**

Cliente 101 ➔ Cidade = Contagem

💡 **Benefício:**
O CDC permite capturar essa alteração sem precisar reler toda a tabela.

Carga completa e incremental

CARGA COMPLETA	CARGA INCREMENTAL
Copia todos os dados	Copia apenas alterações
Mais simples	Mais complexa
Pode consumir mais recursos	Reduz processamento
Útil para carga inicial	Ideal para grandes volumes
Pode reconstruir o destino	Depende de controle da última execução

Exemplo Prático (Tabela com 1 milhão de registros):

- **Primeira execução: ➔ 1.000.000 registros (Carga Inicial Completa)**
- **Próxima execução: ➔ somente registros novos ou modificados (Incremental)**

Carga completa e incremental

Como identificar o que mudou?

- Uma estratégia incremental precisa de algum mecanismo para identificar a mudança.

💧 Marca d'água — Watermark

Uma coluna pode indicar até onde os dados já foram processados.

Exemplo de colunas comuns:
- **id_pedido**
- **data_atualizacao**
- **timestamp**

Lógica de Execução:

Último processamento:
2026-09-20 23:59:59

Próxima execução:
buscar dados > 2026-09-20 23:59:59

🔢 Identificador Crescente

Usar uma chave primária auto-incremental para controlar novas inserções.

Lógica de Execução:

último ID processado = 150000

buscar:
ID > 150000

Carga completa e incremental

Como identificar o que mudou?

CENÁRIO DE MUDANÇA:

1. Pedido 100 ➔ valor = R$ 500
2. Depois: Pedido 100 ➔ valor = R$ 550

🚨 **O Risco:**

Uma estratégia incremental baseada somente em novos IDs pode não perceber essa alteração.

DIFERENCIANDO EVENTOS DE MUDANÇA:

- Novos registros: **INSERT**
- Registros modificados: **UPDATE**
- Registros removidos: **DELETE**

💡 **A Solução:**

É nesse cenário que o CDC — Change Data Capture se torna especialmente útil.

Carga completa e incremental

Streaming x CDC

🌊 STREAMING

Forma de processar dados continuamente.

🔄 CDC

Forma de identificar alterações ocorridas em uma fonte de dados.

PODEMOS TER UMA COMBINAÇÃO COMPLETA:

CDC ➔ Kafka ➔ Streaming ➔ Data Lake

Mas também podemos utilizar CDC sem necessariamente construir todo o processamento como um sistema de streaming complexo.

Padronização e integração de múltiplas fontes

- Imagine três fontes distintas representando dados semelhantes:

POSTGRESQL	API	CSV
id_cliente	customerId	cliente
nome	name	nome_cliente
dt_nascimento	birthDate	nascimento

🚨 **DESAFIO:**
Representam informações semelhantes, mas possuem nomes + formatos + tipos + padrões diferentes.

🔑 **SOLUÇÃO:**
Antes de integrar, precisamos estabelecer uma representação comum.

Padronização e integração de múltiplas fontes

🏷️ **NOMES**

customerId ➔ id_cliente

🔢 **TIPOS**

"150.50" ➔ 150.50

🗓️ **DATAS**

09/21/2026 ➔ 2026-09-21

🔤 **CATEGORIAS**

MG, Minas Gerais, minas gerais ➔ MG

🆔 **IDENTIFICADORES**

Garantir que o mesmo cliente possua um identificador consistente entre as fontes.

Padronização e integração de múltiplas fontes

- Depois da padronização, as fontes podem ser combinadas.

Clientes (PostgreSQL) + Pedidos (API) + Campanhas (CSV)

↓

Dataset Integrado Schema:
[cliente | pedido | data | valor | campanha]

🚀 **RESULTADO:**

Os dados passam a possuir uma estrutura comum e podem seguir para:
Qualidade ➔ Transformação ➔ Armazenamento ➔ BI / IA

Padronização e integração de múltiplas fontes

- Integrar fontes não significa simplesmente realizar um **JOIN**.

É NECESSÁRIO VERIFICAR RIGOROSAMENTE:

- Chaves de relacionamento
- Duplicidades
- Registros sem correspondência
- Tipos de dados
- Fuso horário
- Granularidade
- Regras de negócio
- Dados ausentes
- Atualização das fontes

EXEMPLO DE ENTIDADE DUPLICADA

Um cliente pode existir:
- 1 vez no CRM
- 3 vezes na base de vendas

🎯 **Regra de Negócio:**
Antes de integrar, é necessário determinar se esses registros representam a mesma entidade física ou jurídica.

DÚVIDAS?

OBRIGADO!

Referências bibliográficas

- **DAMJI, Jules S.; WENIG, Brooke; DAS, Tathagata; LEE, Denny.** *Learning Spark: lightning-fast data analytics*. 2. ed. Sebastopol: O'Reilly Media, 2020.

- **KLEPPMANN, Martin.** *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. Sebastopol: O'Reilly Media, 2017.

- **REIS, Joe; HOUSLEY, Matt.** *Fundamentals of data engineering: plan and build robust data systems*. Sebastopol: O'Reilly Media, 2022.

Referências bibliográficas

- **APACHE SOFTWARE FOUNDATION.** *Structured Streaming Programming Guide*. [S. l.]: Apache Spark, [s. d.]. Disponível em: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html. Acesso em: 23 set. 2026.

- **POSTGRESQL GLOBAL DEVELOPMENT GROUP.** *Logical replication*. [S. l.]: PostgreSQL, [s. d.]. Disponível em: Documentação do PostgreSQL — Logical Replication. Acesso em: 23 set. 2026.

- **MONGODB INC.** *Documents*. [S. l.]: MongoDB Documentation, [s. d.]. Disponível em: MongoDB Documentation — Documents. Acesso em: 23 set. 2026.

- **MONGODB INC.** *Data modeling*. [S. l.]: MongoDB Documentation, [s. d.]. Disponível em: MongoDB Documentation — Data Modeling. Acesso em: 23 set. 2026.

Referências bibliográficas

- **DEBEZIUM.** *Debezium Documentation*. [S. l.]: Debezium, [s. d.]. Disponível em: Documentação oficial do Debezium. Acesso em: 23 set. 2026.

- **DEBEZIUM.** *Source connectors*. [S. l.]: Debezium, [s. d.]. Disponível em: Debezium Source Connectors. Acesso em: 23 set. 2026.

- **MICROSOFT.** *Incrementally copy data from a source data store to a destination data store*. Microsoft Learn, 2024. Disponível em: Microsoft Learn — Incremental Copy. Acesso em: 23 set. 2026.

- **MICROSOFT.** *Incrementally copy a table using Azure portal*. Microsoft Learn, 2024. Disponível em: Microsoft Learn — Incremental Copy with Watermark. Acesso em: 23 set. 2026.

Referências bibliográficas

- **AMAZON WEB SERVICES.** *Documentação do AWS Glue*. [S. l.]: AWS, [s. d.]. Disponível em: AWS Glue Documentation. Acesso em: 23 set. 2026.